GitHub Enterprise Practical — Presentation Runbook

0. Before the demo (prep, do this a day ahead)

Accounts needed: 3 GitHub accounts — Owner, Employee, Intern (use a personal + 2 throwaway accounts, or ask 2 teammates to log in on their own laptops/incognito windows).

Tools: Install GitHub CLI (`gh`) and log in as the Owner account for setup:

```bash
gh auth login
```

Enterprise trial (for "Internal" visibility): From github.com → Settings → your profile → "Enterprise" → start a free trial of GitHub Enterprise Cloud, and create/attach your Organization to it. If this isn't feasible in your class's timeframe, skip it and just explain Internal vs Private verbally (see note above).

---

1. Enterprise Setup & Access Control (Owner's screen)

Create the org (web is easier than CLI for org creation): GitHub.com → "+" → New organization → free plan → name it, e.g. `acme-corp-demo`.

Create teams:

```bash
gh api orgs/acme-corp-demo/teams -f name="Full-Time Devs" -f privacy="closed"
gh api orgs/acme-corp-demo/teams -f name="Interns" -f privacy="closed"
```

Add members:

```bash
gh api orgs/acme-corp-demo/teams/full-time-devs/memberships/EMPLOYEE_USERNAME -
gh api orgs/acme-corp-demo/teams/interns/memberships/INTERN_USERNAME -f role="m
```

Create the demo repo:

```bash
gh repo create acme-corp-demo/payments-app --private --clone
```

(If you got the Enterprise trial working, you can instead create it as `--internal`.)

Give teams repo-level permissions:

```bash
gh api -X PUT orgs/acme-corp-demo/teams/full-time-devs/repos/acme-corp-demo/pay
gh api -X PUT orgs/acme-corp-demo/teams/interns/repos/acme-corp-demo/payments-a
```

> Talking point: `maintain` lets employees push branches, manage issues/PRs, and manage some settings but not touch billing/security. `triage` lets interns view code, open issues, and comment/manage issues, but not push code directly — this is your RBAC matrix, shown live.

Show it in the UI: Org → Teams → click each team → Repositories tab → show the permission level next to `payments-app`. This is the "permission matrix" visual your teammate wants.

---

2. Guardrails & Security

Branch protection on `main` (require PRs, require 2 approvals, require status checks):

```bash
gh api -X PUT repos/acme-corp-demo/payments-app/branches/main/protection \
  -H "Accept: application/vnd.github+json" \
  -f required_status_checks[strict]=true \
  -f 'required_status_checks[contexts][]=build' \
  -f enforce_admins=true \
  -f 'required_pull_request_reviews[required_approving_review_count]=1' \
  -f restrictions=null
```

(Do this via the web UI instead if the CLI JSON gets fiddly — Settings → Branches → Add rule — it's more demo-friendly anyway since the audience can see the checkboxes.)

Add a CI check with GitHub Actions — create `.github/workflows/ci.yml`:

```yaml
name: build
on: [pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: echo "running tests..." && exit 0   # flip to exit 1 later to show
```

Push this as the Owner directly (one-time setup, before protection is fully locked), or via a PR.

Add a CODEOWNERS file — `.github/CODEOWNERS`**:**

```
/payments/  @acme-corp-demo/full-time-devs
```

> Talking point: any PR touching `/payments/` automatically requests review from that team — show this by having the intern touch a file in `/payments/` in their PR later and pointing at the auto-requested reviewers.

---

3. Daily Workflow — Intern vs Employee (live, side by side)

As Intern (their own window):

```bash
git clone https://github.com/acme-corp-demo/payments-app.git
cd payments-app
git checkout -b intern/add-discount-logic
# edit a file
git add .
git commit -m "Add discount calculation"
git push origin intern/add-discount-logic
gh pr create --title "Add discount logic" --body "First pass at discount calc"
```

Show the PR landing as "Awaiting review" with the CODEOWNERS auto-reviewer tagged.

As Employee (their own window):

```bash
gh pr view <PR_NUMBER> --web
```

In the web UI, leave an inline comment on a specific line, request a change, then once the intern pushes a fix:

```bash
gh pr review <PR_NUMBER> --approve --body "Looks good, nice work!"
```

---

4. Live Discrepancy Resolution (the climax — rehearse this part!)

Scenario A — Merge Conflict

Set it up beforehand: have both Employee and Intern branch off `main` and edit the *same line* of the same file, merge one first.

```bash
# Intern's branch already merged. Employee now:
git checkout employee/edit-same-line
git merge main
# CONFLICT (content): Merge conflict in app.py
```

Resolve live either in the GitHub web editor (Files changed → "Resolve conflicts" button on the PR) or locally:

```bash
# open app.py, manually pick the correct lines between <<<<<<< / ======= / >>>>>
git add app.py
git commit -m "Resolve merge conflict, keep intern's discount logic"
git push
```

Narrate: employee walks intern through *why* one version was kept.

Scenario B — Accidental Secret Leak

Have the intern (in a throwaway branch) commit a fake key, e.g.:

```bash
echo "AWS_SECRET_KEY=AKIAFAKEEXAMPLE12345" >> config.py
git add config.py && git commit -m "add config" && git push
```

GitHub's push protection / secret scanning should flag it in the PR (works out-of-the-box on public repos; on private repos it needs Advanced Security, which comes with the Enterprise trial). Show the alert in Security → Secret scanning alerts. Then clean it up:

```bash
git revert <bad_commit_sha>
git push
```

For a full history purge (mention this as the "nuclear option" for real leaks): `git filter-repo` or GitHub's "Remove sensitive data" doc — don't actually need to run it live, just reference it.

Scenario C — Breaking Production (Rollback)

Merge a PR with a deliberately broken change into `main` (simulate it slipping through — e.g. temporarily disable branch protection or approve it quickly), then:

```bash
gh pr list --state merged --limit 1
```

On that merged PR in the web UI, click **Revert** — this opens a new auto-generated PR that undoes the change. Merge that PR:

```bash
gh pr merge <REVERT_PR_NUMBER> --merge
```

Narrate: this is the "one-click rollback," production is back to last stable state.

---

5. Presentation Flow (order to actually walk through on stage)

1. **Kickoff (2 min):** Open GitHub Projects board, show 3 tasks already assigned to Owner/Employee/Intern — "this is the whole lifecycle, all inside GitHub."
2. **Access control (3–4 min):** Org → Teams → permission matrix (Section 1).
3. **Guardrails (3–4 min):** Branch protection rule screen, CI check, CODEOWNERS file (Section 2).
4. **Workflow (4–5 min):** Split windows — Intern opens PR → Employee reviews/approves (Section 3).
5. **The 3 discrepancies (8–10 min):** A → B → C, back to back, this is your climax (Section 4).
6. **Wrap-up (1 min):** Move the Projects board card to "Done," close the loop.

Tip: rehearse the timing once fully end-to-end — Scenario A (merge conflict) is the easiest to fumble live, so practice that resolution twice.

---

6. Quick reference — gh CLI cheatsheet used above

Action	Command
Create team	`gh api orgs/ORG/teams -f name="X" -f privacy="closed"`
Add team member	`gh api orgs/ORG/teams/TEAM/memberships/USER -f role="member"`
Set repo permission for team	`gh api -X PUT orgs/ORG/teams/TEAM/repos/ORG/REPO -f permission="maintain"`
Create PR	`gh pr create --title "..." --body "..."`
View PR in browser	`gh pr view N --web`
Approve PR	`gh pr review N --approve --body "..."`
Merge PR	`gh pr merge N --merge`
List merged PRs	`gh pr list --state merged`